

# FastCite: AI-Powered Document Citation and Question-Answering System

A Proposal for Generative AI Course Project

## **Group Members:**

Inam Ullah Shaikh (22i-0857)

Emaan Fatima (22i-0869)

Aden Sial (22i-1313)

Fast National University

November 28, 2025

# Contents

---

|           |                                                     |          |
|-----------|-----------------------------------------------------|----------|
| <b>1</b>  | <b>Introduction</b>                                 | <b>3</b> |
| <b>2</b>  | <b>Problem Statement</b>                            | <b>3</b> |
| <b>3</b>  | <b>Objectives</b>                                   | <b>3</b> |
| 3.1       | Primary Objectives . . . . .                        | 3        |
| 3.2       | Secondary Objectives . . . . .                      | 4        |
| <b>4</b>  | <b>Proposed Solution</b>                            | <b>4</b> |
| 4.1       | System Overview . . . . .                           | 4        |
| 4.1.1     | Document Processing Pipeline . . . . .              | 5        |
| 4.1.2     | Retrieval Augmented Generation (RAG) Flow . . . . . | 5        |
| <b>5</b>  | <b>Expected Outcomes</b>                            | <b>5</b> |
| 5.1       | Functional Deliverables . . . . .                   | 5        |
| 5.2       | Technical Success Metrics . . . . .                 | 6        |
| 5.3       | Learning Goals . . . . .                            | 6        |
| <b>6</b>  | <b>Methodology</b>                                  | <b>6</b> |
| 6.1       | Development Approach . . . . .                      | 6        |
| <b>7</b>  | <b>System Features</b>                              | <b>7</b> |
| 7.1       | User Capabilities . . . . .                         | 7        |
| 7.2       | Admin Capabilities . . . . .                        | 7        |
| <b>8</b>  | <b>Challenges and Mitigation Strategies</b>         | <b>7</b> |
| 8.1       | Key Challenges . . . . .                            | 7        |
| <b>9</b>  | <b>Timeline and Milestones</b>                      | <b>8</b> |
| <b>10</b> | <b>Conclusion</b>                                   | <b>8</b> |
| <b>11</b> | <b>References</b>                                   | <b>8</b> |

# Introduction

---

In the era of digital documentation and academic research, the ability to quickly extract information, ask questions, and obtain accurate citations from large document collections has become increasingly crucial. Traditional methods of searching through documents are time-consuming and often inefficient, especially when dealing with extensive PDF libraries, research papers, and academic materials.

FastCite proposes to address these challenges by developing an intelligent, AI-powered system that leverages state-of-the-art Retrieval Augmented Generation (RAG) techniques. Unlike existing tools, FastCite supports not only textbooks but also **lecture slides, presentations, lecture notes, seminar handouts, and any other PDF learning resources**. Users can ask questions across all types of PDFs and receive answers grounded in source-aware AI with automatic citation generation.

## Problem Statement

---

Modern researchers, students, and professionals face the following challenges when working with large PDF collections:

1. **Inefficient Information Retrieval:** Keyword-based search fails to capture semantic meaning.
2. **Time-Consuming Manual Review:** Users manually read hundreds of pages or slides.
3. **Lack of Context Understanding:** Tools cannot synthesize information from different PDF types.
4. **Citation and Source Tracking:** Users must manually maintain citations from books, slides, or other PDFs.
5. **Scalability Issues:** Real-time processing of long or media-rich PDFs is computationally expensive.

FastCite provides an AI-driven alternative that retrieves semantically relevant content from both long-form textbooks and structured slides, generating answers with accurate page-level citations.

## Objectives

---

### 3.1 Primary Objectives

1. Build a document processing pipeline supporting **textbooks, lecture slides, presentations, lecture notes, and all general PDFs**.
2. Implement semantic vector search using Qdrant to retrieve the most relevant sections from any uploaded PDF.
3. Deploy a Retrieval Augmented Generation (RAG) QA system producing accurate answers with citations from **books, slides, or any PDF source**.

4. Design an intuitive React + TypeScript interface for AI chat, document upload, and citation display.
5. Ensure scalability using asynchronous workers (Celery) to process PDFs efficiently.
6. Provide an admin dashboard for system monitoring and content management.

### 3.2 Secondary Objectives

1. Enable structure-aware chunking optimized for **textbook paragraphs and slide bullet groups**.
2. Support multi-PDF querying across mixed content types (books + slides + notes).
3. Maintain session-based chat history and context awareness.
4. Provide real-time feedback for PDF embedding and processing tasks.
5. Implement user authentication (JWT) with optional 2FA.

## Proposed Solution

---

### 4.1 System Overview

FastCite is a unified web platform capable of processing and reasoning over all PDF study resources including:

- Textbooks, academic papers, and PDF reports
- **Lecture slides, business presentations, seminar decks**
- **Bullet-point lecture notes and handouts**
- **Scanned or image-based PDFs using OCR fallback**

The system consists of:

- **Frontend:** React 18+, TypeScript, TailwindCSS, and chat UI
- **Backend:** FastAPI (Python 3.11+) with async endpoints
- **Vector Search:** Qdrant for semantic similarity + hybrid search
- **Metadata DB:** MongoDB with async driver (Motor)
- **File Storage:** Backblaze B2 for original PDFs
- **Task Queue:** Redis-backed Celery workers for background processing
- **AI Models:** Sentence-Transformers (embeddings) + Google Gemini (generation)

#### 4.1.1 Document Processing Pipeline

PDFs are processed asynchronously through these stages:

1. **Upload & Validation:** Accept any PDF type and store metadata (category: book/slide/note/general PDF).
2. **Text Extraction:** Extract text using PyMuPDF. If empty or scanned, apply OCR fallback.
3. **Structure-Aware Chunking:**
  - Chapter-based semantic chunks for textbooks
  - Group-preserving bullet clusters for slides/notes
  - Heading-aware segmentation
4. **Embedding Generation:** Batch vector embedding using Sentence-Transformers (e.g., all-MiniLM-L6-v2).
5. **Vector Storage:** Store in Qdrant with page numbers, section headers, and document type for citation tracking.
6. **Status Tracking:** Update Celery task progress to user UI.

#### 4.1.2 Retrieval Augmented Generation (RAG) Flow

1. Embed user query using same embedding model.
2. Retrieve top-K relevant chunks via vector + keyword hybrid search from Qdrant.
3. Filter contexts using LLM relevance scoring.
4. Generate final answer using Google Gemini.
5. Extract citation metadata from retrieved chunks.
6. Deliver answer with citations and source page/slide snippets.

## Expected Outcomes

---

### 5.1 Functional Deliverables

1. Users can upload and query **textbooks, slides, notes, presentations, or any PDF**.
2. AI answers grounded in retrieved content with **correct page or slide-section citations**.
3. Real-time processing feedback via Celery worker updates.
4. Admin dashboard for oversight of all uploaded PDFs including slide decks.
5. Conversations preserved per multi-PDF chat session.

## 5.2 Technical Success Metrics

1. Full RAG pipeline correctly retrieving slide + book contexts.
2. Ability to index large PDFs containing **hundreds of pages or slides**.
3. Hybrid semantic + keyword search working across structured and unstructured PDFs.
4. Accurate citation mapping using stored metadata.

## 5.3 Learning Goals

1. Gained experience comparing embedding strategies for slides vs textbooks.
2. Hands-on practice building a scalable multi-PDF RAG QA system.
3. Full-stack AI development using modern async frameworks and vector DBs.

# Methodology

---

## 6.1 Development Approach

### Phase 1: Design & Setup (Week 1-2)

- Architecture design, DB schema, API design, UI mockups

### Phase 2: Core Backend (Week 3-5)

- FastAPI setup, JWT auth, MongoDB models, PDF upload, embedding pipeline

### Phase 3: RAG Engine (Week 6-7)

- Qdrant integration, hybrid semantic search, citation pipeline, Gemini integration

### Phase 4: Frontend (Week 8-9)

- React app, dashboard, chat UI, citation viewer, real-time updates

### Phase 5: Async & Optimization (Week 10)

- Celery workers, task status updates, retry logic, indexing and caching optimization

### Phase 6: Admin Panel & Testing (Week 11-12)

- Admin dashboard, logs, metrics, unit + integration + E2E testing

# System Features

---

## 7.1 User Capabilities

1. Upload any PDF (**slides, books, lecture notes, presentations, reports**)
2. Start document-aware chat sessions
3. Ask natural language questions across **multiple PDFs at once**
4. Receive answers with **page and section-level citations**
5. View referenced PDF or slide snippets
6. Authentication with JWT + optional 2FA

## 7.2 Admin Capabilities

1. System stats across **all PDF types including slides**
2. User role and activity monitoring
3. Delete or audit any uploaded PDF
4. Chat log oversight
5. Processing queue monitoring

# Challenges and Mitigation Strategies

---

## 8.1 Key Challenges

1. **Mixed PDF Structure:** Books vs slides (bullets + images)
  - Apply structure-aware chunking
  - OCR fallback for scanned slide PDFs
2. **Embedding Latency:** Large PDFs generate many embeddings
  - Batch embeddings + background workers
3. **Scalability:** Multiple users + multiple PDFs
  - Async FastAPI + vector DB indexing + task queuing
4. **Citation Accuracy:**
  - Store exact page + section metadata, verify mapping at response time
5. **API Rate Limits (Gemini):**
  - Rate limiting, request queueing, exponential backoff, caching

## Timeline and Milestones

---

| Week  | Milestone                                   |
|-------|---------------------------------------------|
| 1–2   | System Design and Setup Complete            |
| 3–5   | Core Backend Infrastructure Functional      |
| 6–7   | RAG System Implemented and Tested           |
| 8–9   | Frontend Application Complete               |
| 10    | Asynchronous Processing Integrated          |
| 11–12 | Testing, Refinement, Documentation Complete |

Table 1: Project Timeline

## Conclusion

---

FastCite showcases a real-world use-case of Generative AI and RAG systems, enabling seamless citation and question-answering over **textbooks, lecture slides, presentations, notes, and all PDFs**. The modular, asynchronous, and semantic-driven design ensures scalability and accuracy.

## References

---

1. Lewis, P., et al. (2020). “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.” *NeurIPS*, 33.
2. Reimers, N., & Gurevych, I. (2019). “Sentence-BERT: Sentence Embeddings using Siamese BERT Networks.” *EMNLP-IJCNLP*.
3. Karpukhin, V., et al. (2020). “Dense Passage Retrieval for Open-Domain Question Answering.” *EMNLP*.
4. Devlin, J., et al. (2018). “BERT: Pre-training of Deep Bidirectional Transformers.”
5. Vaswani, A., et al. (2017). “Attention is All You Need.”
6. FastAPI Documentation. <https://fastapi.tiangolo.com/>
7. Qdrant Vector Database. <https://qdrant.tech/>
8. Google Gemini API. <https://ai.google.dev/>
9. Celery: Distributed Task Queue. <https://docs.celeryproject.org/>
10. MongoDB Documentation. <https://www.mongodb.com/docs/>